ChatGPT

Federated Multi-Omic Cancer Prognosis (MuFFLe++) – Research & Development Plan

Executive Summary

This plan outlines a hackathon-ready project to build **MuFFLe++**, an advanced federated learning framework for multi-omic cancer prognosis. Our goal is to integrate diverse molecular and clinical data across hospitals without sharing raw records, predicting patient survival or recurrence risk. Key motivations include the need for privacy-preserving collaboration and improved predictive power via multi-omics. We target cancers with rich omics data (e.g. breast, head & neck, bladder), and incorporate genomics (mutations/CNVs), transcriptomics, methylation, proteomics, and clinical covariates. Data will come from public sources like TCGA, CPTAC, ICGC, GEO, and specialized datasets (e.g. CHIMERA for bladder cancer recurrence)【21†L21-L24】【36†L56-L61】.

We will use a **client-server federated architecture** (via NVIDIA FLARE on Nebius GPUs) employing algorithms like FedAvg/FedProx for global model aggregation【41†L43-L52】【45†L23-L30】. Each modality has its own encoder network, and embeddings are fused into a joint feature space. A survival analysis head (e.g. DeepSurv-style Cox network or discrete-time hazard model) predicts time-to-event【44†L623-L627】【30†L778-L785】. To handle data heterogeneity and missing modalities, we adopt selective modality dropout (like MuFFLe's masked-encoder strategy【2†L478-L482】) and personalization (e.g. FedBN or local fine-tuning). Privacy will be enforced via secure aggregation (encrypted updates【2†L471-L476】) and optional differential privacy.

Compute-wise, we leverage Nebius's NVIDIA Blackwell/H100 GPUs with a cost-efficient plan (limited GPUs, mixed precision, mini-batching). The software stack uses Python, PyTorch/TensorFlow, NVIDIA FLARE SDK【48†L61-L64】, and containerized deployments (Docker) for reproducibility. Data preprocessing handles normalization, missing data, and feature alignment across sites. Model evaluation uses concordance index (C-index), time-dependent AUC, and Brier scores, with cross-site (leave-one-hospital-out) validation to simulate deployment. Fairness and calibration analyses will also be included.

We will simulate federated clients locally (e.g. partition TCGA by institution or synthetic FedGen data【1†L329-L338】) for testing. A step-by-step timeline (with milestones for data prep, model training, FL integration, and evaluation) is provided below. We include unit/integration testing strategies, risk mitigation (e.g. data insufficiency, overfitting), and hackathon deliverables (demo notebook, slides, code repo). Tables compare dataset options, model architectures, and privacy mechanisms. Mermaid diagrams illustrate the system architecture and project timeline. Throughout, we cite primary sources (TCGA, CPTAC, NVIDIA docs) and prioritize GPU-efficient, cost-conscious choices. Unresolved issues (e.g. exact datasets, hardware limits) are noted as assumptions/questions.

Project Goals

- **Predict Cancer Prognosis in a Federated Setting**: Develop a privacy-preserving model to predict patient survival or recurrence risk using distributed multi-omic data.
- **Integrate Multi-Modal Data**: Fuse genomics, transcriptomics, methylation, proteomics, and clinical variables for improved accuracy【21†L21-L24】【34†L960-L968】.
- **Handle Modality Gaps**: Enable training when some sites lack certain omics (e.g. no sequencing) by masking encoders as in MuFFLe【2†L478-L482】.
- **Generalizable Models**: Ensure the model learns robust, interpretable patterns across hospitals, not site-specific biases【2†L478-L482】【46†L51-L59】.
- **Privacy and Compliance**: Keep raw patient data local, using encrypted update aggregation and optional DP to protect privacy【2†L471-L476】【41†L43-L52】.
- **Resource-Efficient Implementation**: Use Nebius/NVIDIA infrastructure with optimized GPU usage (multi-GPU as needed, FP16, etc.) to fit hackathon constraints.
- **Reproducible Deliverables**: Provide a working prototype (code, demo, documentation) that could be demoed at the hackathon.

Scientific Motivation

Integrating multi-omic data (genomic, transcriptomic, epigenomic, proteomic) with clinical features often yields more accurate cancer prognosis models than single-data models. For example, large efforts like TCGA have shown the value of multi-omics in understanding tumor biology【21†L21-L24】. Federated learning allows collaboration across hospitals and biobanks without centralizing sensitive data【2†L471-L476】【41†L43-L52】. Recent work (e.g. FedGen, OncoLearn, MuFFLe) demonstrates viability of FL in genomics and cancer. MuFFLe in particular showed a federated RNA+clinical model can stratify bladder cancer patients into risk groups【2†L471-L476】【2†L484-L489】. However, high-dimensional multi-omic datasets pose challenges: “multicollinearity, low events-per-variable, and limited interpretability” in traditional models【46†L51-L59】. Neural survival models (e.g. DeepSurv【44†L623-L627】) and dimension-reduction methods (Coxmos【46†L51-L59】) offer solutions. Our project aims to advance this frontier by federating state-of-the-art multi-omic survival models, pushing toward precision oncology with privacy.

Target Cancer Types

We will focus on cancers with rich public multi-omic data and high clinical impact. Likely candidates include:

- **Breast Cancer (TCGA-BRCA, METABRIC)**: Well-studied with transcriptome, methylation, limited proteomics. OncoLearn used BRCA in federated subtyping【2†L425-L433】.
- **Head & Neck Squamous Cell Carcinoma (TCGA-HNSC)**: Multi-omic (RNA, miRNA, CNV, methylation) available【30†L778-L785】, studied by Coxmos.
- **Bladder Cancer (TCGA-BLCA, CHIMERA)**: CHIMERA provides bladder recurrence (H&E, RNA, clinical)【36†L56-L61】; TCGA-BLCA has multi-omics. MuFFLe used bladder recurrence data【2†L484-L489】.
- **Ovarian Cancer (TCGA-HGSOC)**: TCGA has HGSOC with RNA, CNV, methylation【30†L785-L788】; Coxmos case study.
- **Other (e.g. lung, colon)**: Dependent on time.

Focusing on one or two (e.g. BRCA and BLCA) allows demonstration of federated learning across diverse sites. We will clarify which in assumptions.

Multi-Omic Modalities

We plan to include the following data types:

- **Genomics (DNA variants and CNVs)**: Captures tumor mutations and copy-number changes. Many TCGA datasets include somatic SNVs and CNVs. These are key prognostic markers.
- **Transcriptomics (mRNA expression)**: Gene expression profiles reflect tumor activity. TCGA provides RNA-seq for most cancers. Prior FL work (OncoLearn, CHIMERA) used RNA data【2†L463-L472】【36†L56-L61】.
- **Epigenomics (DNA methylation)**: DNA methylation arrays are available in TCGA (e.g. 450K) and offer orthogonal biological signal. For HNSC/HGSOC, Coxmos included methylation profiles【30†L785-L788】.
- **Proteomics**: Protein abundance data (mass-spectrometry) from CPTAC/Proteomics Data Commons【34†L960-L968】 adds functional layer. E.g. CPTAC has proteomes for several cancers (ovarian, breast, etc). We may start without proteomics, but plan to extend if possible.
- **Clinical Data**: Patient age, stage, histology, treatment, etc. Essential for any prognostic model.
- **Other (Optional)**: Metabolomics, imaging (radiology/pathology). Pathology images (as in CHIMERA) are beyond current scope due to complexity, but could be future work.

Why these modalities? Each layer contributes unique prognostic information. Integrating them has shown improved survival prediction【46†L51-L59】【44†L623-L627】. Genomic mutations and CNVs often drive cancer biology; gene expression captures tumor state; methylation reflects epigenetic regulation; proteomics measures functional protein levels. Clinical features incorporate known risk factors. The combination (multi-block) is challenging but yields better accuracy (e.g. Coxmos demonstrated gains on BRCA, HNSC, HGSOC【46†L51-L59】).

Table 1: Candidate Multi-Omic Data Sources

Data Source	Cancer Types	Modalities Included	Size (patients)	Access & Notes
TCGA (GDC Portal)【21†L21-L24】	33 cancer types (e.g. BRCA, HNSC, BLCA, HGSOC)	WES/WGS mutations, CNVs, RNA-seq, miRNA, methylation (450K), limited proteomics (CPTAC)	~8000 tumor-normal pairs per type (20k total)	Open (GDC portal); some data may require dbGAP/ controlled access (but expression/ methylation often open).
CPTAC (Proteomic Data Commons)【34†L960-L968】	Subset of TCGA cancers (e.g. breast, ovarian, colon)	Mass-spec protein expression (~200+ proteins)	~1000+ samples	Open (NCI PDC) with linkage to TCGA IDs【34†L960-L968】.

Data Source	Cancer Types	Modalities Included	Size (patients)	Access & Notes
ICGC (Data Portal)	30+ cancer projects worldwide	Genomic (SNVs, CNVs), some RNA-seq, methylation	Varies (100s–1000s per project)	Data download (some controlled).
METABRIC	Breast Cancer	Gene expression (microarray), clinical, limited CNV	1980 patients 【44†L609-L618】	Open (EGAS) for research, large survival study 【44†L609-L618】.
GEO/ ArrayExpress	Various (cancer subsets)	Transcriptomics (microarray/RNA)	Varies (tens–hundreds)	Public; may need manual curation.
CHIMERA Challenge (AWS Open Data) 【36†L56-L61】	Bladder cancer recurrence	H&E pathology images, RNA-seq, clinical (bladder cancer)	176 patients (task 3) 【2†L484-L489】	Open on AWS (S3 bucket, CC BY-NC-SA). Includes RNA+clinical 【36†L56-L61】.
Simulated (FedGen) 【1†L329-L338】	-	Synthetic genotype & phenotype	Configurable	Use FedGen toolkit to create realistic federated genomic data for testing.

Each source has pros/cons (Table 1). For hackathon demo, TCGA (on AWS Open Data) is easiest for multi-omics with survival. CHIMERA provides a multimodal bladder dataset. We will assume participant can use AWS S3 or Nebius storage similarly. **Questions/Assumptions:** Which cancer(s) should we prioritize? Do we have specific access budgets or must rely on open data? We'll note if special access is needed.

Federated Learning Architecture

We will implement a **client-server FL** model orchestrated by NVIDIA FLARE【48†L61-L64】【41†L43-L52】. In this setup, each participating site (hospital/biobank) is a *client* with local data and compute; a central *server* aggregates model updates. This architecture is widely used (e.g., FedAvg/FedProx algorithms) and supported by FLARE (see Fig. 1).

```
graph LR
    subgraph Clients
      A(Hospital A)
      B(Hospital B)
      C(Hospital C)
    end
    Server(NVIDIA FLARE Server)
    A -- local training --> UpA[Model Updates]
    B -- local training --> UpB[Model Updates]
    C -- local training --> UpC[Model Updates]
```

```
    UpA & UpB & UpC --> Server
    Server -- aggregated model --> Broadcast(Model)
    Broadcast --> A & B & C
    click A "https://www.cancer.gov/ccg/research/genome-sequencing/tcga" "Open
TCGA info"
    ```
```

**Aggregation Algorithm:** We will use Federated Averaging (FedAvg) as a baseline. To handle non-IID data (statistical heterogeneity) across hospitals, FedProx can be applied (adds a proximal term during local training【45†L23-L30】). NVIDIA FLARE has built-in FedAvg and FedProx support【48†L61-L64】【41†L43-L52】. FedOpt (server-side optimizer) is another option. We'll prototype with FedAvg, then experiment with FedProx if convergence issues arise【45†L37-L42】.

**Personalization:** Given potential heterogeneity, we may incorporate personalized strategies. Options include FedBN (keeping batch-norm stats local), local fine-tuning of global model, or meta-learning approaches. For simplicity, initial release will use vanilla FL; if time permits, we'll explore personalization.

**Secure Aggregation:** Model updates from clients will be encrypted during transit. FLARE's workflow ensures weight updates are only seen as aggregated gradients, preventing the server from inspecting individual updates【41†L43-L52】. Optionally, we can use a Secure Aggregation protocol (e.g., cryptographic masking) so the server only sees the sum of updates, further enhancing privacy.

**Differential Privacy (DP):** To guard against model inversion, we can apply DP to updates. For example, clipping gradients at clients and adding Gaussian noise to their updates before sending. This would trade off accuracy for privacy. We will evaluate feasibility: DP-SGD at each client is doable with frameworks like Opacus (PyTorch) or TensorFlow Privacy. Given limited hackathon time, DP may be added as an extension rather than the initial prototype.

## Model Architecture
### Overview
Our model consists of **modality-specific encoders** whose outputs are fused into a joint representation, followed by a survival-analysis output head. This is similar to MuFFLe's design but extended to more modalities ("MuFFLe++"). The general flow is:

1. **Modality Encoders:** Separate networks process each omic type:
   - *Genomic Encoder*: e.g., an MLP for mutation/CNV vectors.
   - *Transcriptomic Encoder*: e.g., 1D CNN or fully-connected net for RNA expression.
   - *Methylation Encoder*: similar to transcriptome.
```

```
   - *Proteomic Encoder*: e.g., FC layers for protein abundances.
   - *Clinical Encoder*: small FC net for numeric/categorical features.

2. **Fusion Layer:** Outputs of all encoders are concatenated (or combined via attention) into a single latent vector. If a modality is missing at a site, we disable its encoder and fill its output with zeros (as MuFFLe did)【2†L478-L482】.

3. **Survival Head:** The fused embedding passes through a survival model. Options include:
   - **DeepSurv** (Cox-net): a feed-forward net that predicts a relative risk; trained with negative log partial likelihood loss【44†L623-L627】. This handles censoring naturally.
   - **DeepHit** or *Discrete-time model*: predicts hazard probabilities in time bins, optimized by log-likelihood.
   - **Cox Proportional Hazards (linear)**: for interpretability.
   - **Random Survival Forest (RSF)** as a baseline (non-deep).
   - For hackathon, DeepSurv is simplest to implement in PyTorch/TensorFlow and has proven performance on genomic data【44†L623-L627】.

We will likely implement *DeepSurv*-style (fully connected with Cox loss). This network can be trained end-to-end on fused features.

### Model Fusion Strategies
- **Early Fusion**: Concatenate all raw features and feed one net. Not ideal due to missing data.
- **Late Fusion**: Keep encoders separate then combine (our plan). Supports modality dropout.
- **Attention**: Could weigh modalities by importance (advanced).

Given limited time, we'll use simple concatenation of encoder outputs. Later, attention or gating could be added to weigh modalities.

### Addressing High-Dimensionality
Omic data are high-dimensional (e.g. 20k genes). To avoid huge models:
- Apply feature selection or dimensionality reduction per modality (e.g. select top 5000 genes by variance).
- Use autoencoder pretraining or PCA to compress features into fewer latent dims.
- Coxmos suggests PLS models for interpretability【46†L51-L59】, but deep nets can implicitly learn latents.

### Model Comparison Table

| Model Type         | Description                                      |
Pros                                | Cons                                |
|--------------------|--------------------------------------------------|-------------------------
```

DeepSurv (NN)	Neural net with Cox loss (nonlinear hazards).	Captures complex interactions; proven on omics【44†L623-L627】.	Black-box; needs hyperparameter tuning.
Random Survival Forest (RSF)	Ensemble of survival trees.	Handles nonlinearity; robust; few assumptions.	Can't easily fuse multi-modal; less accurate on genomic data (per DeepSurv study【44†L623-L627】).
Cox PH (linear)	Standard Cox model (GLM).	Interpretable coefficients.	Assumes linear effects; fails on high-dim【46†L51-L59】.
PLS-Cox (Coxmos)	Partial Least Squares + Cox【46†L51-L59】.	Deals with collinearity; interpretable.	More complex to implement; slower to train.
DeepHit	Deep net hazard model (multi-output).	Flexible (predicts time distributions).	More complex, heavy training.

For prototype, **DeepSurv** is our primary choice due to ease and proven effectiveness on genomic cancer data【44†L623-L627】. We will implement an MLP that outputs a log-risk, trained with standard Cox partial likelihood. Other models (RSF, Cox) can be baselines if time allows.

Privacy and Security Measures
- **Encrypted Updates**: As in MuFFLe, clients will send *encrypted* weight updates to the server【2†L471-L476】. FLARE's workflow uses SSL/TLS for communication and supports secure aggregation to prevent the server from reading individual client deltas【41†L43-L52】.
- **Differential Privacy**: Optionally, apply DP to gradients. We may use a DP-SGD approach (clip gradients to L2 norm, add Gaussian noise) per client. This helps guard against model inversion. It will be tuned to balance privacy budget and accuracy.
- **Secure Enclave (Optional)**: For added security, a trusted execution environment (like AWS Nitro Enclaves or Azure Confidential ML) could be used to run aggregation, but this is likely beyond scope.
- **Authentication/SSL**: Ensure clients and server use mutual TLS to authenticate, preventing unauthorized participation. FLARE supports SSL certificates【41†L43-L52】.
- **Data Governance**: Each hospital only processes its own data. No PHI leaves site. We will follow common FL practice: raw data and preprocessed features remain local.

Compute/Resource Plan
- **Hardware**: Nebius provides NVIDIA H100/Blackwell GPUs. We plan to use 1 GPU on each client (hospital) node. For the central server, a CPU may suffice (aggregation is lightweight), but could use a GPU if doing global validation.
- **Memory**: Multi-omic input can be large. We will limit each modality's input size (e.g. top-2000 genes, 500 proteins, etc) to keep a single GPU viable (~32–48GB). Mixed precision (FP16) will be enabled to reduce memory.
- **Compute Strategy**: For cost-efficiency:

  - Use **mini-batch training** on clients to reduce memory.
  - Profile model size; prune encoders if needed.
  - If hackathon grants small GPU quotas, we may sequentially train clients rather than concurrently.
- **Simulation**: On Nebius, we might simulate multiple “virtual clients” on one multi-GPU machine if actual multi-node setup is impractical. The plan will consider either multiple VMs or a containerized setup for each client.

Cost minimization: Nebius may offer free hackathon credits. Use spot instances if allowed. Keep model and epochs modest.

Software Stack
- **Programming Languages**: Python (for ML code and orchestration).
- **Frameworks/Libraries**:
  - *NVIDIA FLARE (NVFlare)*: Federated learning SDK【48†L61-L64】. We will use its Python API and example workflows.
  - *PyTorch* (or TensorFlow): to build encoders and survival models. FLARE is framework-agnostic【48†L61-L64】. PyTorch is recommended for its ecosystem (Opacus for DP, etc.).
  - *CUDA/cuDNN*: for GPU acceleration.
  - *PyTorch Lightning* or *TensorFlow Keras* (optional) for structured training.
  - *Opacus (PyTorch)* or *TensorFlow Privacy* for DP modules (if implemented).
  - *scikit-learn/pandas/numpy*: data preprocessing.
- **Containerization**: Docker images with all dependencies to ensure reproducibility. Use NVIDIA CUDA base image with Python, torch, NVFlare installed.
- **Orchestration**: SLURM or Kubernetes on Nebius could run multiple client containers. FLARE has its own admin console/CLI for running tasks.
- **Version Control**: GitHub repository for code and Dockerfiles. CI (GitHub Actions) may run unit tests.

Data Preprocessing Pipeline
For each modality at each site:
1. **Data Loading**: Load raw data (e.g. TCGA CSV/tsv for gene expression, mutation calls, etc.) into Pandas or numpy.
2. **Filtering**: Remove features (genes/proteins) with too many missing values or low variance. Remove patients with missing labels.
3. **Normalization**:
   - Gene expression: log-transformation, z-score or quantile normalization.
   - Methylation: Beta-value normalization or M-value transform.
   - Proteomics: log2 intensities, z-score.
   - Clinical: encode categorical (one-hot) and scale continuous (age, etc.).
4. **Harmonization**: Ensure all sites use the same feature space. For example, fix a common set of genes across hospitals (e.g. intersection or union). For federated setting, initial feature harmonization may require sharing of metadata

(not raw data). We may use a small privacy-preserving protocol (FedViz【1†L335-L340】 style) to align features. If complicated, assume pre-agreed feature set.
5. **Missing Data**: Impute missing clinical values with medians or model-specific defaults.
6. **Partitioning**: If simulating, split full dataset into *k* subsets (one per client), ensuring label distribution is realistic. Possibly mimic institutional biases (e.g. one client has more late-stage cases) to test heterogeneity.

We will script this pipeline and containerize it. Checkpoint intermediate data to avoid recomputation.

Evaluation Metrics and Validation Strategy
- **Primary Metric – Concordance Index (C-index)**: Standard for survival models; measures rank correlation between predicted risk and actual survival time. High-level metric to optimize.
- **Time-dependent AUC**: Compute AUC at fixed times (e.g. 1-year, 3-year survival) using binary classification view.
- **Calibration**: Use calibration plots or Brier score to assess how well predicted risks match observed outcomes (e.g. Integrated Brier Score).
- **Fairness**: Evaluate C-index across subgroups (e.g. gender, age groups, hospitals) to detect biases. FL often improves fairness by learning from diverse cohorts.
- **Cross-Site Validation**: Use k-fold leave-one-client-out CV: train on all but one hospital, test on held-out hospital. This simulates deployment to a new site. Each fold, compute metrics.
- **Internal Validation**: Within each training round, use local validation split to avoid overfitting.
- **Confidence Intervals**: Report metrics with 95% CI via bootstrapping or multiple runs.
- **Ablation**: Compare multi-omic model vs single-modality baselines to demonstrate integration gains (e.g. RNA-only vs RNA+clinical vs RNA+mutation+etc).

Example: Coxmos benchmarking (using AUC) showed PLS-based integrative models outperform simpler ones on BRCA/HNSC【46†L51-L59】. We will use similar rigorous metrics.

Simulation and Testing Plan
- **Simulated Federated Clients**: Locally partition TCGA (or METABRIC) data into *n* chunks. E.g. 4 clients with ~100 patients each. Vary distribution heterogeneity by sharding randomly and by clinical covariates. If real data are limited, use the **FedGen** tool to generate synthetic GWAS-like data【1†L329-L338】.
- **FedGen**: This AWS project can create realistic genomic site data with LD structure and imbalance. We may use it to generate synthetic genotype and trait data to pre-test the FL pipeline【1†L329-L338】.

```
- **Local Prototype**: Before cloud deployment, implement and test the FL loop
on a single machine by simulating clients in separate processes. This
“standalone” mode is supported by NVFlare for quick iteration.
- **Performance Benchmark**: Record training time per round, communication
overhead (model size ~MB), memory usage. Ensure it runs within hackathon limits
(likely < few hours of compute).
- **Sanity Checks**: Test that reducing to a centralized model (all data pooled)
yields similar performance, validating the federated version’s correctness.

## Step-by-Step Implementation Timeline

```mermaid

gantt
    title Project Implementation Timeline
    dateFormat  YYYY-MM-DD
    section Data & Setup
    Data collection (TCGA/CPTAC/ECS)       :done, 2026-10-17, 10d
    Data cleaning & harmonization         : 2026-10-27, 7d
    section Modeling
    Build single-modality survival models: 2026-11-03, 7d
    Build multi-modal fusion model       : 2026-11-10, 7d
    section Federated Framework
    Integrate model into FLARE clients   : 2026-11-17, 5d
    Set up central aggregation server    : 2026-11-22, 3d
    Simulate federated training locally   : 2026-11-25, 7d
    section Privacy & Optimization
    Implement secure aggregation & DP    : 2026-12-02, 5d
    Hyperparameter tuning (FedAvg/Prox)  : 2026-12-07, 4d
    section Evaluation & Deployment
    Cross-site validation & metrics      : 2026-12-11, 4d
    Prepare demo & slides                : 2026-12-15, 3d
    Final integration & cleanup          : 2026-12-18, 2d
```

Figure: Proposed Gantt chart for key phases.

Testing, Unit/Integration Checks

- **Unit Tests**: For data loaders (correct dimensions), encoders (forward pass), and loss computation. E.g. test Cox loss on known tiny dataset.
- **Integration Tests**: End-to-end check: train federated model for 1–2 rounds on synthetic data and verify updates reduce loss.
- **Convergence Criteria**: Ensure global loss decreases or C-index improves over rounds. If divergence seen, check learning rate or data issues.
- **Stress Tests**: Simulate an extreme scenario (e.g. one client has 0 labels) to ensure code handles edge cases gracefully (skip updates, etc.).
```

- **Security Tests**: Verify that client data directories are never read by server code. Confirm weight encryption (we'll log a hash of updates before and after secure-agg to ensure they differ).

Continuous integration could run some tests on a small synthetic dataset whenever code is pushed.

Risk Analysis & Mitigation

- **Data Heterogeneity**: Hospitals may have non-IID distributions, causing slow convergence. *Mitigation:* Use FedProx or adaptive learning rates【45†L37-L42】, and perform sensitivity analysis by simulating heterogeneous splits.
- **Insufficient Data**: Some cancers (or sub-cohorts) have few samples. *Mitigation:* Focus on well-represented cancers first. Use data augmentation or transfer learning (e.g. pretrain on pan-cancer).
- **Overfitting**: High-dimensional omic data can overfit. *Mitigation:* Regularization (L1/L2, dropout), early stopping with validation. Use Coxmos insights (e.g. sPLS models) if needed【46†L51-L59】.
- **Compute Limits**: Nebius quotas or GPU memory may be tight. *Mitigation:* Profile and downscale model, use mixed precision, and leverage Nebius support for hackathon.
- **Privacy Compliance**: Ensuring no data leaks is critical. *Mitigation:* Rigorously review code; use FLARE's built-in privacy features【41†L43-L52】; possibly third-party audit of flow.
- **Timeline Overrun**: Hackathon deadlines are strict. *Mitigation:* Prioritize core FL pipeline (model training, data), then add privacy/DP as time allows. Use Agile sprints.

Hackathon Deliverables

- **Code Repository**: Clean, well-documented GitHub repo with code, scripts, Dockerfile. Include README with setup instructions.
- **Live Demo Notebook**: Jupyter notebook or Streamlit demo showing federated training on sample data and evaluation, runnable on Nebius.
- **Slides / Poster**: Briefly explain problem, approach, architecture (use our system diagram), results (sample metrics/plots).
- **Documentation**: A brief "design document" summarizing algorithms and choices (this report).
- **Reproducibility**: Container image or environment YAML. Possibly a Binder link or small sample data for immediate try-out.
- **Videos (Optional)**: Screen recording of running training or UI, if allowed.

Potential Extensions / Publications

- **Incorporate Imaging**: Extend MuFFLe++ to include radiology or histopathology (e.g. use CNN for pathology and integrate). This aligns with CHIMERA aims.
- **Multi-Task Learning**: Jointly predict recurrence and subtype or drug response.
- **Personalized FL**: Research advanced personalization (clustered FL, meta-learning).
- **Real-World Pilot**: Engage a hospital partner to pilot on their data.
- **Publication**: If results are strong, target a conference like ISMB or FL-focused venues. Emphasize novelty of multi-omic FL.

Outstanding Questions & Assumptions

- **Dataset Choices**: Do we need IRB/consent for any data? We assume use of public datasets (TCGA/CPTAC) only.
- **Hackathon Resources**: What exact GPU types/limits does Nebius allocate for teams? We assume at least 1–2 H100 GPUs.
- **Team Expertise**: We assume familiarity with Python, PyTorch, and some FL concepts.
- **Timeframe**: We plan ~5–6 weeks of work; hackathon end date seems Oct 30, 2026【8†L787-L792】. That suggests ~2 months available (maybe allow for documentation & submission).
- **Client Participation**: We assume at least 3 simulated clients. In real deployment, many sites would join; we will design for ~5–10 as a scalable goal.
- **Data Access**: We assume no costly licenses; all tools used are open source.

References

- NVIDIA FLARE documentation【48†L61-L64】【41†L43-L52】.
- TCGA Program (NCI)【21†L21-L24】.
- Coxmos survival modeling (benchmarking multi-omic)【46†L51-L59】【30†L778-L785】.
- MuFFLe (CMU–NVIDIA hackathon)【2†L471-L476】【2†L478-L482】.
- CHIMERA dataset (AWS)【36†L56-L61】.
- DeepSurv (Yale 2018)【44†L623-L627】.
- FedProx paper【45†L23-L30】【45†L37-L40】.
- Proteomics/CPTAC (NCI)【34†L960-L968】.
- FedGen synthetic data【1†L329-L338】.
- Others (standard FL/SURV references cited in context).

All content above is synthesized from primary sources and literature to ensure accuracy. Detailed citations are provided inline.
